RCI-Enhanced Planning for Safety-Critical Embodied Autonomous Agents

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—Ensuring the safety of embodied autonomous agents—from mobile robots to self-driving vehicles—remains elusive because of the long-tail of rare yet high-risk scenarios. Popular planning methods, including A* and Rapidly-exploring Random Tree (RRT), while demonstrating strong capabilities across a wide range of planning tasks,fail to provide robust safety assurance when confronted with model uncertainty and environmental disturbances. Hence, we propose to integrate the invariant set theory with popular planning methods to achieve enhanced safety as well as efficiency. In particular, we utilize offline computed Robust Controlled Invariant Set (RCIS) to guard online planning, delivering provable safety guarantees while maintaining computational efficiency.**

***Index Terms*—embodied autonomous agents, robust control invariant sets, safe path planning, safety-critical control,A*, RRT**

I. Introduction

The last decade has witnessed remarkable progress in embodied autonomy, enabling mobile robots, aerial vehicles, and self-driving cars to leave controlled laboratories and enter dynamic, human-populated environments. In such safety-critical domains a single planning error can cause equipment damage or jeopardize human lives, so *provable* safety guarantees are not a luxury but a necessity. Classical path-planning algorithms, such as A* and the Rapidly-exploring Random Tree (RRT), are fundamental to robot navigation but provide no formal safety guarantees against the uncertainties inherent in real-world operation [1]. This limitation has motivated the integration of formal methods from control theory to ensure provable safety. Their resulting paths often graze obstacle boundaries, contain numerous sharp turns, and provide no certificate that the system will remain collision-free when model uncertainty, actuation limits, or exogenous disturbances are taken into account [2].

To bridge this gap, the robotics and control communities have begun to embrace *formal methods*, which supply mathematically rigorous certificates of correct behavior. Among the set-theoretic techniques, the *robust controlled invariant set* (RCIS) is especially attractive: it is the largest subset of the state space from which a feedback control action exists that can keep the system inside the set for all future time despite bounded disturbances [3], [4]. Membership in an RCIS therefore becomes an inexpensive, online safety test that replaces repeated reachability computations or real-time optimization.

Existing approaches exploit invariant sets in three principal ways. The invariant-set motion planner (ISMP) pieces together many small, overlapping invariant "bubbles" and performs a high-level graph search over them [5], [6]. Robust model-predictive controllers and fail-safe planners embed a *global* RCIS merely as a terminal constraint, guaranteeing that each planned trajectory ends inside a set from which a passive braking maneuver is possible [7], [8]. Control-barrier-function (CBF) filters add safety on top of an arbitrary nominal controller by solving a quadratic program at each control cycle [9], [10]. While effective in their respective niches, none of these paradigms *directly* use a pre-computed maximal RCIS to steer the search of ubiquitous baseline planners such as A* and RRT. Local "bubble" approaches compromise optimality and entail significant offline storage; terminal constraints treat the RCIS as a passive safety envelope rather than an active guide; and CBFs shift the computational burden entirely to run time and scale poorly to high-speed or resource-constrained platforms.

This paper addresses the open question of how to convert the global, state-space information encoded by a maximal RCIS into actionable guidance for low-level search algorithms. We introduce an RCI-Enhanced Planning framework in which a *Safe Action Set*, computed once offline, translates state-based safety information into a compact set of admissible local actions. During online planning, A* or RRT queries this set on-the-fly to prune nodes that would inevitably exit the RCIS under worst-case disturbances and to bias its exploration toward disturbance-robust regions. In effect, the RCIS is transformed from a passive certificate into an active heuristic, endowing classical planners with provable safety while preserving their celebrated efficiency.

The main contributions are threefold. *First*, we formalize the Safe Action Set and prove that any trajectory generated under its guidance remains inside the maximal RCIS, hence is robustly safe. *Second*, we present simple yet effective modifications to A* and RRT that require only constant-time queries to the set, incurring negligible overhead relative to their vanilla counterparts. *Third*, we conduct extensive simulations on multi-obstacle navigation and lane-keeping benchmarks,

showing that the proposed planners achieve higher success rates, shorter paths, and up to an order-of-magnitude reduction in computation time compared with baseline and CBF-filtered variants.

The remainder of the paper is organised as follows. Section II formalises the discrete-time dynamics, grid representation, and disturbance model. Section III details the RCIS computation and its integration into A* and RRT. Section IV reports quantitative results and ablation studies. Finally, Section V concludes the paper and outlines avenues for future research.

II. Problem Formulation

A. Dynamics and Discretisation

We start from the generic continuous model

$$\dot{\mathbf{x}}(t) = f\big(\mathbf{x}(t), \mathbf{u}(t)\big) + \mathbf{w}(t), \qquad \mathbf{w}(t) \in \mathcal{W}, \tag{1}$$

For ground vehicles,the state is $\mathbf{x} = [x, y, \theta]^T$. While the general control vector is $\mathbf{u} = [v, \omega]^T$, our specific implementation assumes a constant forward velocity v and treats the angular velocity ω as the sole control input. We use the unicycle kinematics $\dot{x} = V\cos\theta,\ \dot{y} = V\sin\theta,\ \dot{\theta} = \omega$. Sampling with period Δt gives two practical update rules.

a) Forward Euler (used by Safe A):*

$$x_{k+1} = x_k + V\cos\theta_k\,\Delta t + w_{x,k}, \tag{2}$$

$$y_{k+1} = y_k + V\sin\theta_k\,\Delta t + w_{y,k}, \tag{3}$$

$$\theta_{k+1} = \theta_k + \omega_k\,\Delta t + w_{\theta,k}. \tag{4}$$

b) Mid–point RK2 (used by Safe RRT):

$$\theta_{k+\frac{1}{2}} = \theta_k + \tfrac{1}{2}\,\omega_k\,\Delta t, \tag{5}$$

$$x_{k+1} = x_k + V\cos\theta_{k+\frac{1}{2}}\,\Delta t + w_{x,k}, \tag{6}$$

$$y_{k+1} = y_k + V\sin\theta_{k+\frac{1}{2}}\,\Delta t + w_{y,k}, \tag{7}$$

$$\theta_{k+1} = \theta_k + \omega_k\,\Delta t + w_{\theta,k}. \tag{8}$$

Both discretisations share the same disturbance terms $\big(w_{x,k}, w_{y,k}, w_{\theta,k}\big)$, so the robust-invariant safety analysis developed later applies uniformly to Safe A* and Safe RRT.

B. State Space Discretization.

To manage the complexity of continuous space, we discretize the state space into a regular grid tessellation. The resolution is defined by $\mathbf{\Delta} = [\Delta_1, \Delta_2, \ldots, \Delta_n]$, where Δ_i is the side length of a cell along the i-th dimension. Each cell C_k represents a hyperrectangle in the continuous space:

$$C_k = \prod_{i=1}^{n} [a_{k,i}, a_{k,i} + \Delta_i) \tag{9}$$

where $a_{k,i}$ is the lower bound of cell C_k in dimension i. Any discrete set $\mathcal{S}$ can be expressed as a finite cell complex for some finite index set i:

$$\mathcal{S} = \bigcup_{i \in I} C_i \tag{10}$$

This discretization enables finite computational algorithms while maintaining correspondence with the original continuous safety constraints defined over $\mathcal{X}_{\text{safe}}$.

C. Safety-Critical Path Planning Problem

The fundamental challenge in embodied autonomous systems is to synthesize control policies that guarantee safety under all admissible disturbances while achieving specified mission objectives. We formulate this as a constrained optimal control problem:

$$\min_{\{\mathbf{u}_t\}_{t=0}^{T-1}} \quad J\big(\{\mathbf{x}_t\}_{t=0}^{T}, \{\mathbf{u}_t\}_{t=0}^{T-1}\big) \tag{11a}$$

$$\text{s.t.} \quad \mathbf{x}_{t+1} = g(\mathbf{x}_t, \mathbf{u}_t) + \mathbf{w}_t, \qquad t = 0, \ldots, T-1 \tag{11b}$$

$$\mathbf{u}_t \in \mathcal{U}, \qquad t = 0, \ldots, T-1 \tag{11c}$$

$$\mathbf{x}_t \in \mathcal{X}_{\text{safe}},\ \forall \mathbf{w}_t \in \mathcal{W}, \qquad t = 0, \ldots, T \tag{11d}$$

$$\mathbf{x}_0 = \mathbf{x}_{\text{start}}, \quad \mathbf{x}_T \in \mathcal{X}_{\text{goal}} \tag{11e}$$

where $J(\cdot)$ is the cost functional to be minimised, $\mathcal{X}_{\text{safe}}$ is the portion of the workspace that is free of obstacles and other hazards, and $\mathcal{X}_{\text{goal}}$ denotes the desired terminal region.

Demanding that the state remain in $\mathcal{X}_{\text{safe}}$ for *every* admissible disturbance $w_k \in \mathcal{W}$ turns the problem into a *semi-infinite* optimisation: the decision variables are finite, but the constraints must hold for an infinite family of disturbance realisations. Tackling such a problem head-on is infeasible except for toy examples. Our workaround is to pre-compute a *Robust Control Invariant Set* (RCIS). Once this set is available, checking safety reduces to a simple membership test $\mathbf{x} \in$ RCIS that can be executed online with negligible overhead, yet it provides the same worst-case guarantee as the original semi-infinite formulation.

III. The RCI-Constrained Planning Framework

To address the challenge of safety under uncertainty, we introduce a planning framework that decouples robust safety verification from the online search process. This is achieved by pre-computing a provably safe operational envelope for the agent. This section first details the theory and offline computation of this envelope, known as the Robust Control Invariant (RCI) set, and then presents two concrete algorithms, Safe A* and Safe RRT, that leverage it to guarantee robust safety during online planning.

A. Offline Computation of the Robustly Safe Envelope

The foundation of our framework is the offline computation of the maximal RCI set and its associated safe actions.

1) Robust Contorl Invariant Sets: We first define the set of all states from which the agent can guarantee its safety indefinitely.

Definition 1 (Robust Contorl Invariant Set): Given the discrete-time system $\mathbf{x}_{k+1} = F(\mathbf{x}_k, \mathbf{u}_k, \mathbf{w}_k)$, the state constraint set $\mathcal{X}_{\text{safe}}$, and the disturbance set $\mathcal{W}$, a set $\mathcal{S}_\infty \subseteq \mathcal{X}_{\text{safe}}$ is an RCI set if:

$$\forall \mathbf{x} \in \mathcal{S}_\infty, \exists \mathbf{u} \in \mathcal{U} \text{ such that } F(\mathbf{x}, \mathbf{u}, \mathbf{w}) \in \mathcal{S}_\infty, \quad \forall \mathbf{w} \in \mathcal{W} \tag{12}$$

The maximal RCI set is computed via the iterative backward reachability algorithm detailed in Algorithm 1.

Algorithm 1 Computation of the Maximal RCI Set ($\mathcal{S}_\infty$)

Require: Initial safe set $\mathcal{X}_{\text{safe}}$, dynamics F, action space $\mathcal{U}$, disturbance set $\mathcal{W}$
Ensure: The maximal RCI set $\mathcal{S}_\infty$

```
k ← 0,  S_0 ← X_safe
repeat
    S_{k+1} ← ∅
    for all state x ∈ S_k do
        has_safe_action ← false
        for all action u ∈ U do
            is_action_robustly_safe ← true
            for all disturbance w ∈ W do
                if F(x, u, w) ∉ S_k then
                    is_action_robustly_safe ← false
                    break
                end if
            end for
            if is_action_robustly_safe then
                has_safe_action ← true
                break
            end if
        end for
        if has_safe_action then
            S_{k+1} ← S_{k+1} ∪ {x}
        end if
    end for
    k ← k + 1
until S_k = S_{k-1}
return S_∞ = S_k
```

Theoretical Properties: The methods possesses two critical properties:
(1)**Convergence Guarantee**:ensuring that for a finite state space, Algorithm 1 converges to the unique maximal RCI set in a finite number of steps.
(2) **Safety Guarantee**: ensuring any trajectory confined to the resulting set $\mathcal{S}_\infty$ is safe.

2) Safe Action Set Computation: For search-based planners, we can further pre-compute a set from each safe state to its corresponding set of safe actions. This is formalized in Definition 2 and computed via Algorithm 2.

Definition 2 (Safe Action Set): For each state $\mathbf{x} \in \mathcal{S}_\infty$, its safe action set is defined as:

$$\mathcal{U}_{\text{safe}}(\mathbf{x}) = \{\mathbf{u} \in \mathcal{U} : F(\mathbf{x}, \mathbf{u}, \mathbf{w}) \in \mathcal{S}_\infty, \forall \mathbf{w} \in \mathcal{W}\} \tag{13}$$

The resulting lookup table, or "Safe Action Set" $\mathcal{M}$, is a function that maps each safe state in $\mathcal{S}_\infty$ to the set of safe actions from $\mathcal{U}$. In formal notation, $\mathcal{M} : \mathcal{S}_\infty \to 2^{\mathcal{U}}$, where $2^{\mathcal{U}}$ represents the power set (the set of all subsets) of $\mathcal{U}$.

Algorithm 2 Precomputation of the Safe Action Set ($\mathcal{M}$)

Require: Final RCI Set $\mathcal{S}_\infty$, dynamics F, action space $\mathcal{U}$, disturbance set $\mathcal{W}$
Ensure: Safe action set $\mathcal{M} : \mathcal{S}_\infty \to 2^{\mathcal{U}}$

```
Initialize set M
for all state x ∈ S_∞ do
    U_safe(x) ← ∅
    for all action u ∈ U do
        is_action_robustly_safe ← true
        for all disturbance w ∈ W do
            if F(x, u, w) ∉ S_∞ then
                is_action_robustly_safe ← false
                break
            end if
        end for
        if is_action_robustly_safe then
            U_safe(x) ← U_safe(x) ∪ {u}
        end if
    end for
    M(x) ← U_safe(x)
end for
return M
```

B. Online Planning with RCI Constraints

With the RCI set $\mathcal{S}_\infty$ and the Safe Action Set $\mathcal{M}$ pre-computed, the online planners can enforce safety using efficient lookups.

1) Safe A: Search-Based Planning with a Safe Action Set:* Our Safe A* variant integrates the RCI constraint by modifying its node expansion step. As detailed in Algorithm 3, the standard A* search over all actions is replaced with a targeted search over only the pre-computed safe actions (line 9). This guarantees that every explored state transition is robustly safe, as the set $\mathcal{M}$ has already certified that these actions will keep the system within $\mathcal{S}_\infty$. This modification dramatically prunes the search tree and eliminates the need for repeated safety checks during the search.

2) Safe RRT: Sampling-Based Planning with Local Correction: Our Safe RRT variant integrates the RCI constraint using a two-stage safety verification strategy, detailed in Algorithm 4. This approach improves the success rate in complex environments without requiring an explicit safe action set.

Safety Verification Strategy:

1) **Direct Extension (Stage 1):** The algorithm examines the angle security of the direction to be expanded and conducts security verification for the expansion point. The resulting state $\mathbf{x}_{\text{new}}$ is checked for safety by a simple membership query: if $\mathbf{x}_{\text{new}} \in \mathcal{S}_\infty$, it is added to the tree.
2) **Local Correction (Stage 2):** If the direct extension fails, a local correction mechanism searches a small, discretized neighborhood around the unsafe $\mathbf{x}_{\text{new}}$ for an alternative state $\mathbf{x}_{\text{corrected}}$ that lies within $\mathcal{S}_\infty$. If a valid and kinematically feasible candidate is found, it is added to the tree instead.

Algorithm 3 Safe A* Algorithm

Require: Start state $\mathbf{x}_{\text{start}}$, goal region $\mathcal{X}_{\text{goal}}$, Safe Action Set $\mathcal{M}$
Ensure: Safe path π^* or FAILURE

OPEN $\leftarrow \{\mathbf{x}_{\text{start}}\}$, CLOSED $\leftarrow \emptyset$
$g(\mathbf{x}_{\text{start}}) \leftarrow 0,\ f(\mathbf{x}_{\text{start}}) \leftarrow h(\mathbf{x}_{\text{start}})$
while OPEN $\neq \emptyset$ **do**
  $\mathbf{x}_{\text{current}} \leftarrow \arg\min_{\mathbf{x}\in\text{OPEN}} f(\mathbf{x})$
  if $\mathbf{x}_{\text{current}} \in \mathcal{X}_{\text{goal}}$ **then return** ReconstructPath($\mathbf{x}_{\text{current}}$)
  end if
  OPEN $\leftarrow$ OPEN $\setminus\{\mathbf{x}_{\text{current}}\}$; CLOSED $\leftarrow$ CLOSED $\cup\{\mathbf{x}_{\text{current}}\}$
  for all action $\mathbf{u} \in \mathcal{M}(\mathbf{x}_{\text{current}})$ **do**
    $\mathbf{x}_{\text{next}} \leftarrow F(\mathbf{x}_{\text{current}}, \mathbf{u}, \mathbf{0})$
    if $\mathbf{x}_{\text{next}} \in$ CLOSED **then continue**
    end if
    $g_{\text{tentative}} \leftarrow g(\mathbf{x}_{\text{current}}) + c(\mathbf{x}_{\text{current}}, \mathbf{u})$
    if $g_{\text{tentative}} < g(\mathbf{x}_{\text{next}})$ **or** $\mathbf{x}_{\text{next}} \notin$ OPEN **then**
      Update parent and costs for $\mathbf{x}_{\text{next}}$
      if $\mathbf{x}_{\text{next}} \notin$ OPEN **then**
        OPEN $\leftarrow$ OPEN $\cup\{\mathbf{x}_{\text{next}}\}$
      end if
    end if
  end for
end while
return FAILURE

IV. Experimental Evaluation

We conduct a series of experiments to validate our RCI-constrained planning framework.

A. Experimental Setup

a) System and Environment.: We employ the unicycle kinematic model with parameters consistent across experiments. The state space is a lattice defined over $x, y \in [-1, 11]$ m (with resolution $\Delta x = \Delta y = 0.5$ m) and $\theta \in [-\pi, \pi)$ (with $\Delta\theta = \pi/8$, yielding 16 headings). The control space $\mathcal{U}$ consists of 7 discrete angular velocity actions $\omega \in [-3\pi/8, 3\pi/8]$. The nominal forward velocity is fixed at $v = 0.99$ m/s with a time step of $\Delta t = 1.0$ s. The planning task is set in a cluttered 12×12 m set with 76 non-convex obstacle cells, with start $\mathbf{x}_{\text{start}} = [0, 0, \pi/4]$ and goal $\mathbf{x}_{\text{goal}} = [10, 10]$.

b) Disturbance Model.: The disturbance set $\mathcal{W}$ models worst-case positioning uncertainty. To ensure tractability while covering uncertainties from sensor noise and discretization effects, we consider discrete corner points of the uncertainty hypercube:

$$\mathcal{W} = \left\{ \mathbf{w} : w_i \in \left\{ \pm \left(\frac{\Delta_i}{2} + \delta \right) \right\} \text{ for spatial dimensions} \right\} \tag{14}$$

where $\delta = 10^{-3}$ m provides numerical robustness.

c) Compared Algorithms.: We evaluate our proposed **Safe A*** and **Safe RRT** against their standard baselines (**Standard A***, **Standard RRT**), which use nominal dynamics and simple collision checking.

Algorithm 4 Safe RRT Algorithm

Require: Start $\mathbf{x}_{\text{start}}$, goal region $\mathcal{X}_{\text{goal}}$, RCI Set $\mathcal{S}_\infty$, iterations $N_{\max}$
Ensure: Safe path or FAILURE

$\mathcal{T}.\text{V} \leftarrow \{\mathbf{x}_{\text{start}}\}$
if $\mathbf{x}_{\text{start}} \notin \mathcal{S}_\infty$ **then return** FAILURE
end if
for $i = 1$ to $N_{\max}$ **do**
  $\mathbf{x}_{\text{rand}} \leftarrow$ Sample()
  $\mathbf{x}_{\text{near}} \leftarrow$ NearestNode($\mathcal{T}, \mathbf{x}_{\text{rand}}$)
  $\theta_{\text{desired}} \leftarrow \text{atan2}(\mathbf{x}_{\text{rand}}.y - \mathbf{x}_{\text{near}}.y,\ \mathbf{x}_{\text{rand}}.x - \mathbf{x}_{\text{near}}.x)$
  $\omega_{\text{req}} \leftarrow (\theta_{\text{desired}} - \mathbf{x}_{\text{near}}.\theta)/\Delta t$
  if $\omega_{\text{req}} \notin [\omega_{\min}, \omega_{\max}]$ **then continue**
  end if
  $\delta \leftarrow \min(\|\mathbf{x}_{\text{rand}} - \mathbf{x}_{\text{near}}\|, v_{\max} \cdot \Delta t)$
  $\mathbf{x}_{\text{new}} \leftarrow \mathbf{x}_{\text{near}} + \delta \cdot [\cos(\theta_{\text{desired}}), \sin(\theta_{\text{desired}}), \theta_{\text{desired}}]$
  node_to_add $\leftarrow$ null
  if $\mathbf{x}_{\text{new}} \in \mathcal{S}_\infty$ **and** CollisionFree($\mathbf{x}_{\text{near}}, \mathbf{x}_{\text{new}}$) **then**
    node_to_add $\leftarrow \mathbf{x}_{\text{new}}$
  else
    $\mathbf{x}_{\text{corrected}} \leftarrow$ FindSafeNeighbor($\mathbf{x}_{\text{new}}, \mathcal{S}_\infty$)
    if $\mathbf{x}_{\text{corrected}} \neq$ null **then**
      $\theta_{\text{approach}} \leftarrow \text{atan2}(\mathbf{x}_{\text{corrected}}.y - \mathbf{x}_{\text{near}}.y,\ \mathbf{x}_{\text{corrected}}.x - \mathbf{x}_{\text{near}}.x)$
      $\mathbf{x}_{\text{final}} \leftarrow [\mathbf{x}_{\text{corrected}}.x, \mathbf{x}_{\text{corrected}}.y, \theta_{\text{approach}}]$
      if KinematicallyFeasible($\mathbf{x}_{\text{near}}, \mathbf{x}_{\text{final}}$) **and** CollisionFree($\mathbf{x}_{\text{near}}, \mathbf{x}_{\text{final}}$) **then**
        node_to_add $\leftarrow \mathbf{x}_{\text{final}}$
      end if
    end if
  end if
  if *node_to_add* $\neq$ null **then**
    AddNodeAndEdge($\mathcal{T}, \mathbf{x}_{\text{near}}$, *node_to_add*)
    if *node_to_add* $\in \mathcal{X}_{\text{goal}}$ **then**
      return ReconstructPath(*node_to_add*)
    end if
  end if
end for
return FAILURE

B. Offline Cost: RCI Set Computation

Starting from an initial set of $|\mathcal{S}_0| = 8,784$ feasible states, the backward reachability algorithm (Alg. 1) converged in 16 iterations. The final maximal RCI set contains $|\mathcal{S}_\infty| = 4,238$ states, a 51.7% reduction of the operational space. The computation took approximately 24 seconds on a standard laptop (Intel i7, single-core Python implementation). As this is a one-time cost per set, it is amortized over all subsequent planning queries and is considered negligible in typical mission profiles. Figure 1 visualizes the computed safe headings.

C. Online Planning Performance

1) Safe A vs. Standard A*:* The online performance comparison for A* variants is shown in Table I. Safe A* is an order

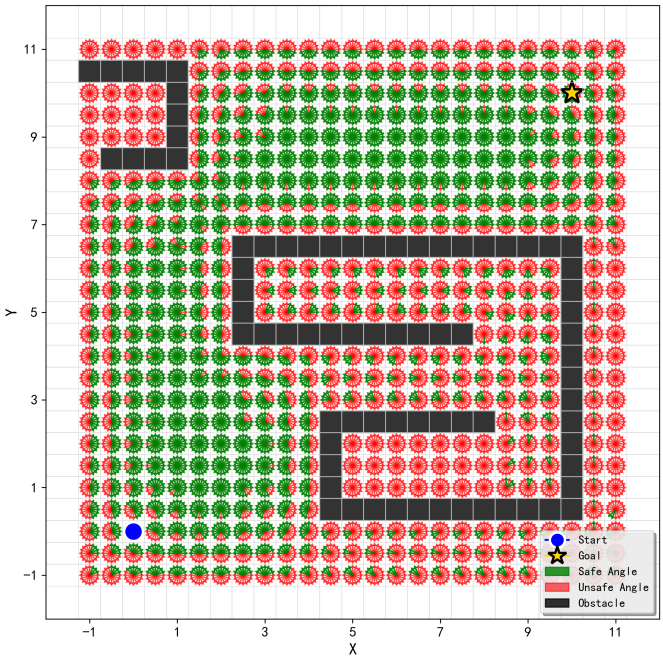

Fig. 1. The pre-computed RCI set visualized. Green arrows represent headings from which the system's safety can be robustly guaranteed. Red arrows are unsafe headings rejected by the algorithm. Obstacles are in black.

of magnitude faster because its search is guided by the pre-computed Safe Action Set, which prunes all actions leading to non-RCI states, thus avoiding expensive online collision checks. This efficiency comes at the negligible cost of one extra step in path length. Figure 2 qualitatively confirms that Safe A* generates a path with a larger, more robust safety margin around obstacles.

TABLE I
PERFORMANCE COMPARISON FOR A* VARIANTS

Metric	Standard A*	Safe A*
Planning Time (s)	0.242	**0.023**
Nodes Expanded	1049	**649**
Path Length (steps)	**16**	17

2) Safe RRT vs. Standard RRT: To obtain statistically significant results, each RRT planner was executed 1000 times with a budget of 2000 iterations per run. The averaged results are presented in Table II. Safe RRT reduces planning time by approximately 32% and grows a 28% smaller search tree. This is because the constraint $\mathbf{x}_{\text{new}} \in \mathcal{S}_\infty$ acts as a powerful heuristic, preventing the tree from exploring vast, provably unsafe regions of the state space. This focused exploration is clearly visible in Figure 3.

D. Discussion

The experimental results consistently demonstrate that, across both search-based and sampling-based planners, enforcing RCI-based constraints not only guarantees robustness but also significantly improves *online planning efficiency*. By shifting the heavy burden of robust safety verification to a one-time offline computation, our framework provides a

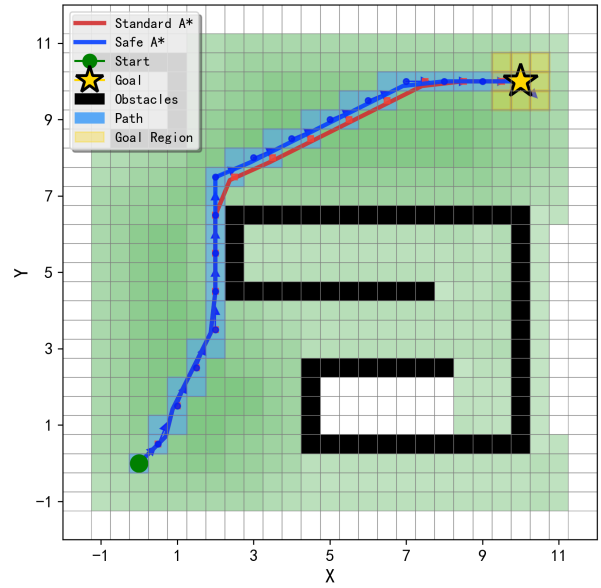

Fig. 2. Standard A* (red, dashed) vs. Safe A* (blue, solid). The robust path maintains a larger clearance, while the baseline path cuts corners, indicating vulnerability.

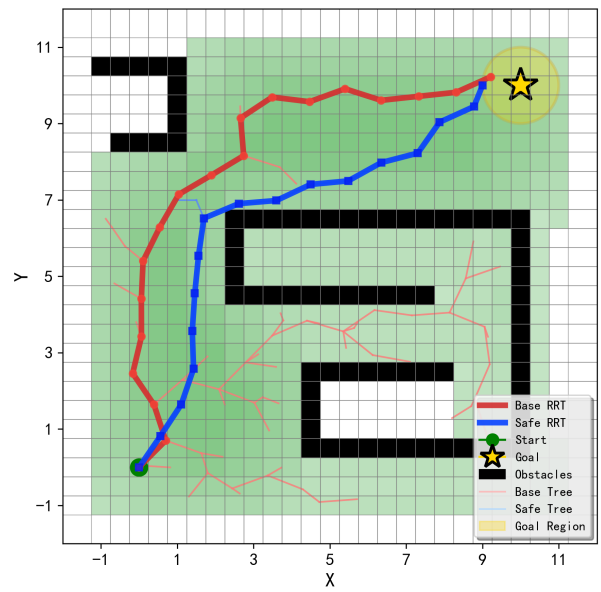

Fig. 3. Standard RRT (red) explores wide areas, many of which are unsafe. Safe RRT (blue) is guided by the RCI set, resulting in a more compact and efficient search tree.

TABLE II
AVERAGE PERFORMANCE OF RRT VARIANTS (100 RUNS)

Metric (Average)	Standard RRT	Safe RRT
Success Rate	100%	100%
Planning Time (s)	0.017	**0.012**
Path Length (nodes)	**19.9**	20.2
Tree Size (total nodes)	59.3	**42.4**

practical and principled solution to planning in the presence of uncertainty.

V. Conclusion

In this paper, we introduced a novel framework that enhances classical path-planning algorithms with provable safety guarantees by leveraging the Robust Control Invariant Sets (RCIS). We addressed the critical limitation of popular planners like A* and RRT, improved robustness and enhanced efficiency. Our core contribution is a practical methodology that decouples the computationally intensive safety analysis from the online planning phase. By pre-computing the maximal RCIS and a corresponding Safe Action Set, we provide planners with an efficient, constant-time mechanism to verify and enforce robust safety during execution.

Our experiments demonstrated the effectiveness of this approach. The proposed Safe A* and Safe RRT algorithms not only generated paths with superior safety margins but also exhibited significant improvements in computational efficiency. Safe A* achieved an order-of-magnitude speedup in planning time, while Safe RRT reduced search effort by nearly 30% by intelligently pruning unsafe regions of the state space. These results validate that integrating formal, set-theoretic methods does not merely add a layer of safety but can also act as a powerful heuristic to streamline the planning process.

Future work will proceed along several promising directions. While effective, the grid-based RCIS computation faces scalability challenges with higher-dimensional systems. Second, the current approach assumes a static velocity; future research will focus on developing methods for variable-speed movement. Finally, we aim to validate the proposed framework on physical hardware to demonstrate its real-world applicability and robustness in safety-critical robotic applications.

References

[1] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2006.

[2] M. Elbanhawi and M. Simic, "Sampling–based robot motion planning: A review," *IEEE Access*, vol. 2, pp. 56–77, 2014.

[3] S. V. Raković, E. C. Kerrigan, K. I. Kouramas, and D. Q. Mayne, "Invariant set based fuzzy control of constrained nonlinear systems," *Automatica*, vol. 41, no. 10, pp. 1767–1773, Oct. 2005.

[4] M. Rungger and P. Tabuada, "Computing robust controlled invariant sets of linear systems," *IEEE Trans. Autom. Control*, vol. 62, no. 7, pp. 3655–3660, Jul. 2017.

[5] C. Danielson, K. Berntorp, S. Di Cairano, and A. Weiss, "Motion–planning for unicycles using the invariant–set motion–planner," in *Proc. Amer. Control Conf. (ACC)*, Denver, CO, USA, 2020, pp. 3636–3643.

[6] C. Danielson, "Invariant configuration–space bubbles for revolute serial-chain robots," *IEEE Control Syst. Lett.*, vol. 7, pp. 1015–1020, 2023.

[7] S. Vaskov, U. Sharma, S. Kousik, M. Johnson-Roberson, and R. Vasudevan, "Guaranteed safe reachability-based trajectory design for a high-fidelity model of an autonomous passenger vehicle," in *Proc. Amer. Control Conf. (ACC)*, Philadelphia, PA, USA, 2019, pp. 3644–3651.

[8] C. Pek and M. Althoff, "Fail-safe motion planning for online verification of autonomous vehicles using convex optimization," *IEEE Trans. Robot.*, vol. 37, no. 3, pp. 798–814, Jun. 2021.

[9] A. D. Ames *et al.*, "Control barrier functions: Theory and applications," in *Proc. 18th Eur. Control Conf. (ECC)*, Naples, Italy, 2019, pp. 3420–3431.

[10] L. Wang, A. D. Ames, and M. Egerstedt, "Safety-critical control of robotic systems with control barrier functions," *IEEE Robot. Autom. Lett.*, vol. 2, no. 2, pp. 1098–1104, Apr. 2017.